

Міністерство освіти і науки України
НТУУ «Київський політехнічний інститут ім. Ігоря Сікорського»
Навчально-науковий Фізико-технічний інститут

Системна інженерія

Індивідуальний проект

об'єктно-орієнтована модель
Пивоварні

Виконав:
Студент 2 курсу НН ФТІ
групи ФБ-42

Бойко Євгеній

Перевірив:
Кіфорчук К. О.

Мета: побудова об'єктно-орієнтованої моделі системи на основі специфікації на мові SysML.

Система - Пивоварня

1.Опис системи

Пивоварня може виконувати складний технологічний цикл: вмикатися, налаштовуватися за заданим рецептом, молоти солод, затирати, фільтрувати та кип'ятити сусло, охолоджувати його, проводити ферментацію (бродіння), освітлювати пиво, розливати у тару, виконувати санітарну обробку (CIP-мийку) та вимикатися.

Пивоварня не працює сама по собі: виробничим процесом керує оператор – Пивовар, який є обов'язковим атрибутом системи. Крім того, система має складну структуру (включає підсистеми помелу, варильного порядку, ферментації тощо) та повинна суворо контролювати свій технологічний стан, щоб унеможливити порушення послідовності дій (наприклад, неможливо почати варіння сусла, якщо сировина ще не підготовлена).

Зафіксуємо означене у вигляді нового класу:

```
class Brewery:

    def __init__(
        self,
        brewer,
        brew_kettle_volume,
        bottling_capacity,
        fermenter_count,
        power_consumption,
        min_work_temperature,
        max_work_temperature,
    ):
        self.brewer = brewer
        self.brew_kettle_volume = brew_kettle_volume
        self.bottling_capacity = bottling_capacity
        self.fermenter_count = fermenter_count
        self.power_consumption = power_consumption
        self.min_work_temperature = min_work_temperature
        self.max_work_temperature = max_work_temperature
        self.state = "off"

        self.power_supply_system = PowerSupplySystem()
        self.control_system = ControlAutomationSystem()
```

```

self.milling_system = MillingSystem()
self.mashing_system = MashingSystem()
self.wort_filtration_system = WortFiltrationSystem()
self.boiling_system = BoilingSystem()
self.thermal_processing_system = ThermalProcessingSystem()
self.fermentation_system = FermentationSystem()
self.final_filtration_system = FinalFiltrationSystem()
self.bottling_packaging_system = BottlingPackagingSystem()
self.sanitation_system = SanitationSystem()

self.subsystems = {
    "power_supply": self.power_supply_system,
    "control_system": self.control_system,
    "milling_system": self.milling_system,
    "mashing_system": self.mashing_system,
    "wort_filtration_system": self.wort_filtration_system,
    "boiling_system": self.boiling_system,
    "thermal_processing_system": self.thermal_processing_system,
    "fermentation_system": self.fermentation_system,
    "final_filtration_system": self.final_filtration_system,
    "bottling_packaging_system": self.bottling_packaging_system,
    "sanitation_system": self.sanitation_system,
}

def require_state(self, *allowed):
    if self.state not in allowed:
        raise InvalidStateError(
            f"Поточний стан '{self.state}' не дозволяє цю операцію."
        )

def power_on(self):
    self.require_state("off")
    self.power_supply_system.supply_power()
    self.state = "waiting"

def select_recipe(self, recipe_name, mash_temperature,
fermentation_temperature, pressure):
    self.require_state("waiting")
    self.control_system.set_recipe(recipe_name, mash_temperature,
fermentation_temperature, pressure)
    self.state = "recipe_selected"

def prepare_raw_materials(self):
    self.require_state("recipe_selected")
    self.control_system.start_control()
    self.milling_system.process_malt()
    self.state = "raw_material_preparation"

def brew_wort(self):
    self.require_state("raw_material_preparation")
    params = self.control_system.get_parameters()
    temperature = params["mash_temperature"]

```

```

        self.mashing_system.mash(target_temperature=temperature)
        self.wort_filtration_system.filter_wort()
        self.boiling_system.boil_with_hops(boil_temperature=100.0)
        self.state = "wort_brewing"

    def cool_and_ferment(self):
        self.require_state("wort_brewing")
        params = self.control_system.get_parameters()
        temperature = params["fermentation_temperature"]

        self.thermal_processing_system.cool_wort()
        self.fermentation_system.ferment(fermentation_temperature=temperature)
        self.state = "fermentation_and_maturation"

    def filter_and_finish(self):
        self.require_state("fermentation_and_maturation")
        self.final_filtration_system.clarify_beer()
        self.state = "final_filtration"

    def bottle_and_package(self):
        self.require_state("final_filtration")
        self.bottling_packaging_system.bottle_and_package()
        self.control_system.log_parameters()
        self.state = "packaged"

    def clean_equipment(self):
        self.require_state("packaged")
        self.sanitation_system.clean()
        self.state = "cleaned"

    def power_off(self):
        self.require_state("waiting", "recipe_selected", "cleaned")
        self.control_system.stop_control()
        self.power_supply_system.disconnect_power()
        self.reset_for_next_cycle()
        self.state = "off"

    def reset_for_next_cycle(self):
        for subsystem in self.subsystems.values():
            subsystem.reset()

    def get_status(self):
        return {
            "brewery": self.state,
            **{name: subsystem.state for name, subsystem in
self.subsystems.items()}},
        }

```

2.Опис акторів

Виробничим процесом пивоварні керує пивовар(на основі абстракції Людина):

```

class Person:
    """Клас, що описує людину."""

    def __init__(self, name, age):
        self.name = name
        self.age = age

class Brewer(Person):
    """Клас, що описує пивовара."""

    def __init__(self, name, age, employee_id, qualification):
        super().__init__(name, age)
        self.employee_id = employee_id
        self.qualification = qualification

```

3.Опис підсистем

Оскільки система Пивоварні складається з 11 підсистем, які мають спільну початкову поведінку, для уникнення дублювання коду було створено базовий клас BaseSubsystem. Від нього успадковуються всі інші підсистеми.

Описаний базовий клас та всі підсистеми пивоварні:

```

class BaseSubsystem:
    """Базовий клас для всіх підсистем."""

    def __init__(self):
        self.state = "idle"

    def reset(self):
        self.state = "idle"

class PowerSupplySystem(BaseSubsystem):
    """Підсистема енергозабезпечення."""

    def __init__(self):
        super().__init__()
        self.state = "off"

    def supply_power(self):
        self.state = "on"

    def disconnect_power(self):
        self.state = "off"

    def reset(self):
        self.state = "off"

```

```

class ControlAutomationSystem(BaseSubsystem):
    """Підсистема управління та автоматизації."""

    def __init__(self):
        super().__init__()
        self.recipe = None
        self.mash_temperature = None
        self.fermentation_temperature = None
        self.pressure = None

    def set_recipe(self, recipe_name, mash_temperature,
fermentation_temperature, pressure):
        self.recipe = recipe_name
        self.mash_temperature = mash_temperature
        self.fermentation_temperature = fermentation_temperature
        self.pressure = pressure
        self.state = "configured"

    def start_control(self):
        self.state = "running"

    def get_parameters(self):
        return {
            "recipe": self.recipe,
            "mash_temperature": self.mash_temperature,
            "fermentation_temperature": self.fermentation_temperature,
            "pressure": self.pressure,
        }

    def log_parameters(self):
        self.state = "logged"

    def stop_control(self):
        self.state = "stopped"

    def reset(self):
        super().reset()
        self.recipe = None
        self.mash_temperature = None
        self.fermentation_temperature = None
        self.pressure = None

class MillingSystem(BaseSubsystem):
    """Підсистема помелу."""

    def __init__(self):
        super().__init__()
        self.malt_loaded = False
        self.grain_crushed = False

    def load_malt(self):

```

```

        self.malt_loaded = True
        self.state = "malt_loaded"

    def grind_grain(self):
        self.grain_crushed = True
        self.state = "grain_crushed"

    def process_malt(self):
        self.load_malt()
        self.grind_grain()
        self.state = "milled"

    def reset(self):
        super().reset()
        self.malt_loaded = False
        self.grain_crushed = False

class MashingSystem(BaseSubsystem):
    """Підсистема затирання."""

    def __init__(self):
        super().__init__()
        self.ingredients_mixed = False
        self.mash_ready = False
        self.target_temperature = None

    def mix_malt_with_water(self):
        self.ingredients_mixed = True
        self.state = "mixed"

    def heat_mash(self, target_temperature):
        self.target_temperature = target_temperature
        self.state = f"heated_to_{target_temperature}C"

    def saccharify(self):
        self.mash_ready = True
        self.state = "mash_ready"

    def mash(self, target_temperature):
        self.mix_malt_with_water()
        self.heat_mash(target_temperature)
        self.saccharify()
        self.state = "mashed"

    def reset(self):
        super().reset()
        self.ingredients_mixed = False
        self.mash_ready = False
        self.target_temperature = None

class WortFiltrationSystem(BaseSubsystem):

```

```

    """Підсистема фільтрації сусла."""

    def __init__(self):
        super().__init__()
        self.wort_separated = False
        self.grain_sparged = False

    def separate_wort_from_grain(self):
        self.wort_separated = True
        self.state = "wort_separated"

    def sparge_grain(self):
        self.grain_sparged = True
        self.state = "sparged"

    def collect_wort(self):
        self.state = "wort_collected"

    def filter_wort(self):
        self.separate_wort_from_grain()
        self.sparge_grain()
        self.collect_wort()
        self.state = "filtered"

    def reset(self):
        super().reset()
        self.wort_separated = False
        self.grain_sparged = False


class BoilingSystem(BaseSubsystem):
    """Підсистема кип'ятіння."""

    def __init__(self):
        super().__init__()
        self.hops_added = False
        self.boiled = False
        self.boil_temperature = None

    def heat_wort(self, boil_temperature):
        self.boil_temperature = boil_temperature
        self.state = f"heating_to_{boil_temperature}C"

    def add_hops(self):
        self.hops_added = True
        self.state = "hops_added"

    def boil_wort(self):
        self.boiled = True
        self.state = "boiled"

    def boil_with_hops(self, boil_temperature):
        self.heat_wort(boil_temperature)

```



```

        self.add_hops()
        self.boil_wort()
        self.state = "boiling_complete"

    def reset(self):
        super().reset()
        self.hops_added = False
        self.boiled = False
        self.boil_temperature = None

class ThermalProcessingSystem(BaseSubsystem):
    """Підсистема термічної обробки."""

    def __init__(self):
        super().__init__()
        self.cooled = False

    def cool_wort(self):
        self.cooled = True
        self.state = "cooled"

    def reset(self):
        super().reset()
        self.cooled = False

class FermentationSystem(BaseSubsystem):
    """Підсистема ферментації."""

    def __init__(self):
        super().__init__()
        self.yeast_added = False
        self.fermentation_completed = False
        self.matured = False
        self.yeast_separated = False
        self.fermentation_temperature = None

    def add_yeast(self):
        self.yeast_added = True
        self.state = "yeast_added"

    def start_fermentation(self, fermentation_temperature):
        self.fermentation_temperature = fermentation_temperature
        self.state = f"fermenting_at_{fermentation_temperature}C"

    def maintain_temperature(self):
        self.state = "temperature_stable"

    def mature_beer(self):
        self.matured = True
        self.state = "matured"

```

```

def separate_yeast(self):
    self.yeast_separated = True
    self.fermentation_completed = True
    self.state = "fermentation_completed"

def ferment(self, fermentation_temperature):
    self.add_yeast()
    self.start_fermentation(fermentation_temperature)
    self.maintain_temperature()
    self.mature_beer()
    self.separate_yeast()
    self.state = "fermented"

def reset(self):
    super().reset()
    self.yeast_added = False
    self.fermentation_completed = False
    self.matured = False
    self.yeast_separated = False
    self.fermentation_temperature = None

class FinalFiltrationSystem(BaseSubsystem):
    """Підсистема фільтрації та фінішної обробки."""

    def __init__(self):
        super().__init__()
        self.filtered = False

    def filter_beer(self):
        self.filtered = True
        self.state = "filtered"

    def stabilize_beer(self):
        self.state = "stabilized"

    def clarify_beer(self):
        self.filter_beer()
        self.stabilize_beer()
        self.state = "clarified"

    def reset(self):
        super().reset()
        self.filtered = False

class BottlingPackagingSystem(BaseSubsystem):
    """Підсистема розливу та пакування."""

    def __init__(self):
        super().__init__()
        self.containers_prepared = False
        self.filled = False

```

```

        self.packaged = False

    def prepare_containers(self):
        self.containers_prepared = True
        self.state = "containers_ready"

    def fill_containers(self):
        self.filled = True
        self.state = "filled"

    def label_product(self):
        self.state = "labeled"

    def package_batch(self):
        self.packaged = True
        self.state = "packaged"

    def bottle_and_package(self):
        self.prepare_containers()
        self.fill_containers()
        self.label_product()
        self.package_batch()
        self.state = "packaging_complete"

    def reset(self):
        super().reset()
        self.containers_prepared = False
        self.filled = False
        self.packaged = False


class SanitationSystem(BaseSubsystem):
    """Підсистема санітарії."""

    def __init__(self):
        super().__init__()
        self.cleaned = False

    def rinse_equipment(self):
        self.state = "rinsed"

    def disinfect_equipment(self):
        self.state = "disinfected"

    def finish_cleaning(self):
        self.cleaned = True
        self.state = "cleaned"

    def clean(self):
        self.rinse_equipment()
        self.disinfect_equipment()
        self.finish_cleaning()
        self.state = "sanitized"

```

```
def reset(self):
    super().reset()
    self.cleaned = False
```

4. Демонстрація роботи системи.

Створимо об'єкт актора (Пивовара), а потім саму систему (Пивоварню), передавши актора та базові технічні характеристики

```
def demo():
    brewer = Brewer(
        name="Олександр",
        age=20,
        employee_id="BR-101",
        qualification="Старший пивовар",
    )

    brewery = Brewery(
        brewer=brewer,
        brew_kettle_volume=1000,
        bottling_capacity=800,
        fermenter_count=4,
        power_consumption=25,
        min_work_temperature=2,
        max_work_temperature=100,
    )
```

Запустимо послідовний технологічний процес і простежимо за тим, як змінюються стани підсистем та головної системи.

```
print("=== Ініціалізація ===")
print(brewery.get_status())

print("\n=== Увімкнення ===")
brewery.power_on()
print(brewery.get_status())

print("\n=== Вибір рецепта ===")
brewery.select_recipe(
    "Світлий лагер",
    mash_temperature=65.0,
    fermentation_temperature=12.0,
    pressure=1.2
)
print(f"Рецепт: {brewery.control_system.recipe}, стан пивоварні: {brewery.state}")

print("\n=== Підготовка сировини ===")
```

```

brewery.prepare_raw_materials()
print(brewery.get_status())

print("\n=== Варіння сусла ===")
brewery.brew_wort()
print(brewery.get_status())

print("\n=== Охолодження та ферментація ===")
brewery.cool_and_ferment()
print(f"Температура ферментації:
{brewery.fermentation_system.fermentation_temperature}°C")
print(brewery.get_status())

print("\n=== Фінішна фільтрація ===")
brewery.filter_and_finish()
print(brewery.get_status())

print("\n=== Розлив і пакування ===")
brewery.bottle_and_package()
print(brewery.get_status())

print("\n=== Санітарна обробка ===")
brewery.clean_equipment()
print(brewery.get_status())

print("\n=== Вимкнення ===")
brewery.power_off()
print(brewery.get_status())

```

Згідно з вимогами до проекту, у системі реалізовано контроль станів. Наприклад, пивоварня не може розпочати варіння сусла, якщо вона вимкнена або якщо не підготовлена сировина. Спроба виконати варіння сусла (brew_wort), коли систему вже вимкнено:

```

print("\n=== Перевірка контролю станів ===")
try:
    brewery.brew_wort()
except InvalidStateError as error:
    print(f"Помилка: {error}")

```

Вивід:

```

=== Ініціалізація ===
{'brewery': 'off', 'power_supply': 'off', 'control_system': 'idle', 'milling_system': 'idle', 'mashing_system': 'idle', 'wort_filtration_system': 'idle', 'boiling_system': 'idle', 'thermal_processing_system': 'idle'}

=== Ввімкнення ===
{'brewery': 'waiting', 'power_supply': 'on', 'control_system': 'idle', 'milling_system': 'idle', 'mashing_system': 'idle', 'wort_filtration_system': 'idle', 'boiling_system': 'idle', 'thermal_processing_system': 'idle'}

=== Вибір рецепта ===
Рецепт: Світлий лагер, стан пивоварні: recipe_selected

=== Підготовка сировини ===
{'brewery': 'raw_material_preparation', 'power_supply': 'on', 'control_system': 'running', 'milling_system': 'milled', 'mashing_system': 'idle', 'wort_filtration_system': 'idle', 'boiling_system': 'idle', 'thermal_processing_system': 'idle'}

=== Варіння сусла ===
{'brewery': 'wort_brewing', 'power_supply': 'on', 'control_system': 'running', 'milling_system': 'milled', 'mashing_system': 'mashed', 'wort_filtration_system': 'filtered', 'boiling_system': 'boiling_complete', 'thermal_processing_system': 'idle'}

=== Охолодження та ферментація ===
Температура ферментації: 12.0°C
{'brewery': 'fermentation_and_maturation', 'power_supply': 'on', 'control_system': 'running', 'milling_system': 'milled', 'mashing_system': 'mashed', 'wort_filtration_system': 'filtered', 'boiling_system': 'boiling_complete', 'thermal_processing_system': 'idle'}

=== Фінішна фільтрація ===
{'brewery': 'final_filtration', 'power_supply': 'on', 'control_system': 'running', 'milling_system': 'milled', 'mashing_system': 'mashed', 'wort_filtration_system': 'filtered', 'boiling_system': 'boiling_complete', 'thermal_processing_system': 'idle'}

=== Розлив і пакування ===
{'brewery': 'packaged', 'power_supply': 'on', 'control_system': 'logged', 'milling_system': 'milled', 'mashing_system': 'mashed', 'wort_filtration_system': 'filtered', 'boiling_system': 'boiling_complete', 'thermal_processing_system': 'idle'}

=== Санітарна обробка ===
{'brewery': 'cleaned', 'power_supply': 'on', 'control_system': 'logged', 'milling_system': 'milled', 'mashing_system': 'mashed', 'wort_filtration_system': 'filtered', 'boiling_system': 'boiling_complete', 'thermal_processing_system': 'idle'}

=== Вимкнення ===
{'brewery': 'off', 'power_supply': 'off', 'control_system': 'idle', 'milling_system': 'idle', 'mashing_system': 'idle', 'wort_filtration_system': 'idle', 'boiling_system': 'idle', 'thermal_processing_system': 'idle'}

=== Перевірка контролю станів ===
Помилка: Поточний стан 'off' не дозволяє цю операцію.

```